SkyBridge Compass: A Crypto-Agile P2P Handshake Framework for Remote Desktop and File Transfer with Auditable Downgrade Policy and Post-Quantum Readiness

Zi'ang Li, Peng Liu

Abstract—Peer-to-peer remote desktop and file-transfer systems run across heterogeneous devices, untrusted networks, and multi-year lifecycles in which cryptographic assumptions can expire faster than products. Yet "post-quantum readiness" is usually treated as a one-off algorithm swap rather than a migration problem with downgrade resistance, auditability, and reproducibility requirements. We present an *auditable crypto-agile migration contract* for authenticated P2P key exchange. The contract separates negotiated suites, provider backends, and a policy layer; restricts any classic fallback to a local-only, pre-send, rate-limited path so that timeout and network-derived failures can never trigger a downgrade; and emits a minimal, replayable evidence tuple for every downgrade decision. We formalize the contract and *machine-check* it: a Dolev-Yao Tamarin model with an active attacker and long-term and KEM key compromise proves session-key secrecy, injective entity authentication, negotiation integrity, downgrade resistance, weak forward secrecy for v1, and genuine forward secrecy for the v2 hybrid suite (surviving long-term-key compromise). All 13 lemmas verify with no falsifications. The contract is realized as a portable, provider-agnostic core (a Rust implementation with C-ABI and UniFFI bindings) deployed on macOS and iOS, directly answering the "generalize beyond Apple" concern at the mechanism level. On macOS, end-to-end handshake latency is 2.232 ms (classic), 2.743 ms (ML-KEM-768 + ML-DSA-65), and 3.483 ms (v2 hybrid forward secrecy), with payloads of 687–12077 B ($N = 1000$ each). Because the handshake runs once per long remote-desktop session or large transfer, this cost amortizes to a negligible fraction of session time. Reported latencies are deployment-instance measurements; we claim mechanism-level portability, not cross-platform performance parity.

Index Terms—post-quantum cryptography, crypto agility, downgrade resistance, secure handshake, peer-to-peer systems, remote desktop, auditability, reproducibility, platform deployment.

1 Introduction

Peer-to-peer (P2P) remote desktop and file transfer systems must establish secure sessions under adversarial network conditions, device churn, and multi-year lifecycles. These systems face a practical tension: users expect seamless pairing across heterogeneous stacks, while defenders require that cryptographic negotiation remains robust against downgrade and implementation-level failure modes. Post-quantum cryptography (PQC) increases this tension. Larger messages, higher computational cost, and uneven platform support make fallback tempting—yet fallback is exactly where silent downgrade and unverifiable behavior tend to hide.

The technical contribution of this paper is a *generic, crypto-agile P2P authenticated key exchange (AKE) and migration contract*; this is why we compare against generic secure-channel baselines (TLS 1.3, QUIC, DTLS, Noise) throughout. Remote desktop and file transfer are the *deployment* that motivates the contract's constraints rather than a restriction of its scope: both run one handshake per long-lived session or large transfer, so an implementer can afford a deterministic, auditable, downgrade-resistant negotiation whose one-time cost amortizes over the session (Section 7.6).

Existing secure-channel designs and frameworks provide mature cryptographic building blocks, but they do not, by default, solve the migration problem an implementer actually faces: (i) selecting among classic/PQC/hybrid suites across provider/suite strata in heterogeneous runtimes, (ii) ensuring that policy (e.g., "require PQC") is enforced at the entry points where callers cannot accidentally violate it, (iii) targeting deterministic failure semantics under timeouts, reorder/duplication, and malformed inputs, and (iv) emitting security-relevant telemetry that can be audited without relying on ad hoc or excessive logging. In practice, many systems either hard-code a single suite, or allow "compatibility" to creep into negotiation in ways that are hard to reason about and harder to verify. For this paper, "locally generated" fallback causes are narrowly scoped: pre-send capability/provider instantiation failures for already paired peers (pinned trust record); timeout and unauthenticated network-derived failures are never fallback-eligible.

To make the migration gap explicit in reviewer-visible form, Table 1 maps the four requirements above to what widely deployed handshakes provide by default versus what this work adds as a system-level contract.

This paper addresses these gaps with an auditable migration contract, instantiated in SkyBridge Compass, for staged PQC migration and downgrade-auditable negotiation, with Apple platforms as a deployment instance.

Distinction from protocol-plus-policy wrappers: In widely deployed stacks, the causes and semantics of retry/fallback are typically pushed into application glue, making downgrade decisions hard to specify, prove, or audit post hoc. Our contract makes

- *Zi'ang Li is with Independent Researcher, Tianjin, China. E-mail: 2403871950@qq.com.*
- *Peng Liu is with Independent Researcher, Qiqihar, China.*

TABLE 1
Migration requirements and default baseline coverage in Introduction scope.

Req.	Widely deployed defaults (TLS/QUIC/DTLS/Noise)	SkyBridge contract in this paper
(i) Suite/provider migration	Protocol negotiation primitives exist (Noise patterns may be fixed), but runtime provider migration policy is mostly external to the protocol	Explicit separation of suite stratum and provider-backend stratum with capability-ordered selection
(ii) Policy non-bypass at entry points	Typically enforced in caller glue; policy/suite mismatch handling is implementation-specific	Policy-bound handshake sessions plus compile-fail harness + runtime precondition checks (Supplementary, Regression Testing)
(iii) Deterministic fault semantics	Alert/pattern outcomes exist, but fallback semantics are often application-defined	Actor-isolated one-shot state machine with timeout-blocked fallback and deterministic terminal templates
(iv) Auditable security telemetry	Downgrade prevention exists, but event schema and evidence sufficiency are externalized	D3 minimal evidence set + typed events + transcript-anchored in-session hash chain

migration semantics part of the security surface: (D2) timeout and other network-derived failures are never fallback-eligible, so an active adversary, under this default-policy gate and threat model, cannot create a classic downgrade edge by suppression; (D3) any policy-allowed downgrade action emits a minimal replayable evidence tuple sufficient to re-derive "what happened and why" under the pinned trust record and policy mode. The policy-bound, deterministic state machine ensures evidence corresponds to the actual decision path and is validated under fault injection and timeout suppression (Table 6).

The core idea is to treat crypto agility as a layered contract: negotiated suites define the protocol surface; provider backends implement suites using platform-native or portable cryptography; and a policy layer governs fallback and produces downgrade-specific structured evidence only when fallback action is taken (other exceptional states emit typed events but are not labeled as downgrade). Concretely, policy consumes peer-auth result, local capability/provider errors, offered suites, and timer/attempt state, and outputs allowed suites, retry plan, and mandatory event set. We complement this design with an actor-isolated handshake driver that enforces one-shot completion, timeouts, and sensitive-material zeroization, yielding deterministic outcomes under the modeled transport faults. The portability claim here is mechanism-level portability of the migration contract (provider/suite separation, policy semantics, and verification workflow) across heterogeneous stacks, not cross-platform latency parity. Operationally, the strict policy now has two explicit realizations: *Compliance Mode* (no classic path at all) and *Bootstrap-Assisted Mode* (one-time classic control channel only to refresh paired KEM identity keys, followed by mandatory PQC rekey before business traffic is released; control messages are type-restricted to KEM-refresh, and business message types are rejected before key release). We separate mechanism-level claims (policy-gated downgrade behavior, deterministic failure semantics, and auditability) from deployment-level claims (absolute latency/throughput), and evaluate each layer with corresponding evidence.

The contract's guarantees—negotiation integrity, transcript binding, mutual authentication, session-key secrecy, replay resistance, and policy-defined downgrade resistance with auditability—are stated as theorems and *machine-checked* against an active Dolev-Yao attacker in Section 5. Here "deterministic outcomes" means that for a fixed input transcript, policy mode, and fault class the terminal outcome and required event template are unique. Explicit key confirmation uses two Finished frames so that both peers confirm transcript-derived keys before any business data is released (Fig. 1), matching standard AKE key-confirmation notions. The v1 non-claims and limitations are stated in Section 8.

1.1 Contributions

This paper makes three contributions.

1) **C1: An auditable crypto-agile migration contract, backed by a machine-checked security proof.** The contract (a) separates negotiated suites from provider backends with capability-ordered selection; (b) enforces policy non-bypass at the handshake entry points, so callers cannot reintroduce ad hoc fallback; (c) admits only a narrow, local-only, pre-send classic fallback, making timeout- and network-derived failures fallback-ineligible *by construction* (Definition D2); and (d) emits a minimal, replayable downgrade-evidence tuple for every accepted fallback (Definition D3). We give formal definitions and prove the resulting guarantees—session-key secrecy, injective authentication, negotiation integrity, and downgrade resistance—in a Dolev-Yao Tamarin model with an active attacker and long-term/KEM key compromise; all 13 lemmas verify (Section 5; `formal/skybridge_v2.spthy`). As a sub-point, an opt-in v2 hybrid suite composes the static KEM secret with a fresh ephemeral contribution; unlike v1 (which the model shows has only weak forward secrecy), v2 achieves *genuine* forward secrecy that survives long-term-key compromise, also proven in the model (`ForwardSecrecy_v2`). *Evidence:* Section 5; Table 4; Supplementary Table S22.
2) **C2: A portable, provider-agnostic reference implementation.** The contract is implemented as an OS-independent Rust core with a C-ABI binding (for C/C++/C# P/Invoke) and a UniFFI binding (for Kotlin and Swift), so non-Apple clients can drive the same negotiation and downgrade-policy logic from one shared core. This is our concrete answer to "generalize beyond Apple": the mechanism, not just the paper's description, is language- and OS-agnostic. We deploy it on macOS and iOS and are consolidating reference Windows, Android, and Linux clients onto the same core. *Evidence:* Section 6; Supplementary Table S33.
3) **C3: Reproducible benchmarks.** We release artifact pipelines that report distributional metrics (p50/p95/p99, CIs, effect sizes) rather than point estimates, and that reproduce every primary figure and table, including handshake latency, downgrade-policy behavior under fault injection, and session-level amortization. *Evidence:* Fig. 4; Table 10; `AuditTrailTests`; `HandshakeBenchmarkTests`.

1.2 Paper Organization

Section 2 reviews related work on P2P pairing, negotiation, and PQC migration. Section 3 details system architecture and protocol design. Section 4 presents the handshake state machine. Section 5 presents the security model and guarantees. Section 6 describes implementation highlights and platform instantiation.

Section 7 outlines evaluation methodology and metrics. Section 8 discusses limitations and future work, and Section 9 concludes. Supplementary Materials provide full wire-format definitions, implementation details, and extended tables.

2 Related Work

Because our contribution is a generic crypto-agile P2P AKE and migration contract, the natural comparison set is the family of widely deployed authenticated key-exchange and secure-channel designs (TLS 1.3, QUIC, DTLS, Noise) together with PQC-migration guidance; remote desktop and file transfer are the deployment that motivates the contract's constraints, not its boundary. We use the following terms throughout: *crypto agility* (ability to migrate primitives without redesign), *downgrade resistance* (prevent or detect forced fallback), *suite negotiation* (explicit, authenticated selection of handshake KEM/SIG/AEAD/KDF), *PQC readiness* (ability to deploy PQC alongside classic suites), and *auditability* (deterministic, minimal event traces for security decisions). Our threat model assumes an active network attacker attempting to induce fallback or obscure failures; therefore we focus on mechanisms that make negotiation and failures explicit and auditable.

2.1 Peer-to-Peer Device Pairing

Pairing protocols such as Bluetooth SSP [1], Continuity [2], and PAKEs like SPAKE2+ [3] establish initial trust but stop short of post-pairing migration, downgrade policy, or auditability. SkyBridge Compass begins *after* pairing and treats pinned identity keys as inputs rather than outcomes.

2.2 Cryptographic Agility and Negotiation

TLS 1.3 and QUIC bind negotiated parameters into transcripts and transport parameters to prevent silent downgrade [4], [5], but they do not define an auditable, system-level fallback policy across heterogeneous providers. We operationalize migration guidance [6], [7] with explicit policy gates and event emission that make downgrade decisions observable and testable.

2.3 Handshake Frameworks and Noise-Style Patterns

Noise provides a compact framework for describing authenticated key exchange patterns with explicit transcript binding and identity key usage [8]. While SkyBridge Compass does not implement Noise directly, our MessageA/MessageB/Finished exchange targets similar transcript-binding goals while adding explicit suite negotiation and auditable policy outcomes.

2.4 Post-Quantum Cryptography Deployment

PQC standardization and early deployments highlight size and performance costs, along with pressure for hybrid deployments [9], [10], [11], [28], [29], [12], [30], [31], [33], [13], [14], [15]. Recent IETF work standardizes both hybrid X25519MLKEM768 key agreement [42] and standalone ML-KEM named groups [43] for TLS 1.3, reflecting the same static-versus-hybrid migration trade-off our v1/v2 suites address. We focus on making downgrade policy and auditability explicit across providers, treating Apple's CryptoKit PQC [16], [23] as a deployment instance rather than a protocol dependency.

2.5 Secure State Machine Design

State machine vulnerabilities have been a persistent source of security bugs in protocol implementations [17], [18]. Actor-based concurrency models, as implemented in Swift's actor system [19], provide compile-time guarantees against data races. We extend this model with deterministic completion, timeout handling, and audit events for cryptographic handshakes.

2.6 Baseline Comparison with Widely Deployed Handshakes

Table 2 contrasts SkyBridge Compass with widely deployed handshake frameworks on explicit negotiation, downgrade auditability, and failure semantics, and reports message count, wire size, and latency where available.

Baseline protocols expose transcript integrity but do not specify system-level retry causes or replayable downgrade evidence. Detailed baseline methodology (time-to-ready endpoint, warmup policy, ALPN setup, kickoff payload settings, and loopback pcap accounting) and full baseline tables are provided in the Supplementary Materials (notably Table S24).

Importantly, baseline protocols' downgrade protections target negotiated parameter integrity within a single handshake. They do not specify when an implementation should *retry* with weaker cryptography after local failures, nor do they require a minimal, replayable evidence set for such a decision; in practice, these behaviors are pushed into application glue and operational logging. Our migration contract makes the retry/downgrade boundary explicit (Definitions D2–D3) and audit-visible, and we validate it under active suppression (e.g., Fig. 5).

2.7 Novelty Boundary and Incremental Delta

We do not claim novelty in authenticated transcript binding, suite negotiation primitives, or actor isolation by themselves; these are reused. The delta is making *migration semantics* part of the security surface (contribution C1): the contract binds negotiation to a declared provider backend rather than an implicit platform default; enforces policy non-bypass at the entry points so caller glue cannot reintroduce fallback; confines fallback eligibility to a local-only, pre-send cause set that excludes timeout and network-derived failures by definition (D2); attaches a replayable, downgrade-specific evidence tuple with deterministic event typing that distinguishes downgrade from failure (D3); and realizes strictness in two explicit forms (Compliance, and Bootstrap-Assisted recovery with a mandatory PQC-rekey gate before business traffic). Supplementary Table S1 gives the full prior-art-versus-delta matrix with evidence anchors, and Supplementary Table S9 gives an artifact-backed ablation-by-property map for these mechanisms.

2.8 PQC Migration Design Space

To make migration-risk boundaries explicit, we classify designs on two axes: (1) fallback trigger origin (*network-derived* vs *local-only* vs *never fallback*), and (2) evidence strength for operator-side verification (*none/external log* vs *structured replayable evidence*).

In this space, TLS/QUIC deployment defaults [4], [5] provide strong handshake transcript integrity but leave system-level retry and fallback causes, and any downgrade evidence, implementation-defined and outside the protocol; Signal PQXDH [10] performs hybrid key agreement at the application layer but likewise leaves fallback eligibility and replayable evidence outside the protocol.

TABLE 2
Baseline comparison with widely deployed handshakes. Message counts and negotiation semantics are summarized from the TLS/QUIC/DTLS RFCs and Noise specification [4], [5], [8], [20]. Loopback latencies and wire sizes are measured with BaselineBenchRunner (release), N=1000; wire sizes report loopback p50/p95 (KiB) and are cert-dependent for TLS/QUIC/DTLS. SkyBridge loopback wire size includes framing/transport overhead (and may include SBP1 handshake padding); payload-only handshake bytes are reported in Supplementary Table S14. Noise wire size is pattern-dependent and reflects the XX pattern used in this baseline.

Protocol	Suite nego. explicit	Auditable downgrade policy	Failure semantics	Msgs	Wire size (p50/p95, KiB)	p50/p95 latency
TLS 1.3 [4]	Yes (cipher suites, groups)	Downgrade prevention only; audit external	Alerts defined; audit external	4 (1 RTT)	8.6/8.7	8.53/14.01 ms
QUIC [5]	Yes (TLS 1.3)	Same as TLS 1.3	Same as TLS 1.3	4 (1 RTT)	7.6/14.1	0.34/11.37 ms
WebRTC DTLS [20]	Yes (DTLS 1.2 ciphers)	Downgrade prevention only; audit external	Alerts defined; audit external	4 (1 RTT)	3.0/3.1	8.18/11.79 ms
Noise XX [8]	No (fixed pattern)	No policy layer	Pattern-defined; audit external	3	2.5/2.6	0.44/0.58 ms
					4.8/4.9 (classic)	1.44/1.57 ms (classic)
					36.9/43.6 (liboqs)	1.95/2.76 ms (liboqs)
SkyBridge Compass	Yes (suite + policy bound)	Yes (explicit policy + events)	Deterministic + evented	4	18.6/18.8 (CryptoKit)	5.91/7.53 ms (CryptoKit)

SkyBridge occupies the corner that combines a local-only fallback whitelist with a timeout/network-derived blacklist and structured, replayable downgrade evidence with deterministic event typing. Supplementary Table S2 provides the full matrix.

3 System Architecture

3.1 Overview

SkyBridge Compass implements a layered architecture separating concerns across four primary domains: discovery, cryptographic operations, handshake management, and session transport. Fig. 2 illustrates the high-level component relationships and trust boundaries.

3.2 Threat Model

We model an active network attacker in the Dolev-Yao style. The attacker fully controls the discovery channel: it can drop, delay, reorder, and duplicate packets; modify or inject arbitrary bytes in MessageA, MessageB, or the Finished frames; replay prior handshake messages across sessions; spoof capability or policy claims to induce weaker negotiation; and force negotiation failures by corrupting suites, key shares, or signatures. Extending the baseline model, the attacker may also compromise a peer after pairing and reveal its long-term signing and KEM private keys; the proofs in Section 5 are established against this stronger reveal-capable attacker. Each capability is met by a specific protocol or policy mechanism, summarized in Table 3: transcript binding via `sigB`, responder-side negotiation-integrity checks, nonce-derived replay control, the policy-gated downgrade rules (no timeout- or network-derived fallback; strict policy is either Compliance Mode or a control-only Bootstrap-Assisted Mode with mandatory PQC rekey), per-peer rate limiting, and trust-record gating of legacy P-256 acceptance.

We trust that the out-of-band pairing ceremony delivers the correct peer identity key (e.g., visual PIN comparison on both screens, with no blind trust), that the device Keychain preserves the integrity of stored identity keys, and that the Secure Enclave, when available, provides hardware-backed key isolation resisting software-level extraction. Out of scope are traffic analysis from size/timing metadata (SBP2 is an optional mitigation evaluated in Section 7 without changing the core claims), side-channel and physical attacks on cryptographic implementations or Secure Enclave hardware, compromise of the operating-system kernel, and social engineering of users.

TABLE 3
Attacker capabilities and the mechanism that addresses each.

Attacker capability	Mechanism (protocol + policy)
Modify/inject handshake bytes	Transcript binding: `sigB` covers MessageA and the chosen suite; edits to `supportedSuites[]`, `keyShares[]`, or `policy` fail verification
Spoof/select unoffered suite	Negotiation integrity: Responder must pick an offered suite with a matching key share, else `missingKeyShare`
Replay prior handshakes	`handshakeId` from fresh nonces; duplicates rejected within a window
Force timeouts to induce fallback	Downgrade resistance: timeout/network-derived fallback disallowed (D2); strict policy is fail-closed or control-only bootstrap
Rapid downgrade cycling	Per-peer fallback cooldown (default 5 min)
Stranger forcing legacy P-256	Legacy acceptance gated on authenticated channel or trust record

3.3 Protocol Message Flow

Fig. 1 illustrates the handshake message sequence and transcript coverage. Supplementary Table S10 enumerates suite-negotiation outcomes under the two-attempt strategy.

MessageA carries the `supportedSuites[]` (offered suites) and `keyShares[]` arrays (up to two entries to bound size); MessageB selects a suite and binds MessageA via `sigB`. Both sides verify that the selected suite was offered and that a matching key share exists, preventing negotiation mismatch. Under the two-attempt strategy (Section 6.7), PQC and classic are not co-offered in a single `MessageA`; when fallback is permitted, the Initiator retries with a classic-only `MessageA` after a whitelist of locally generated capability and negotiation failures. Under strict policy, an optional Bootstrap-Assisted recovery path may transiently establish a classic-only control channel to refresh paired KEM identity keys, but business traffic remains gated until a successful PQC rekey. Explicit key confirmation uses two small Finished frames to avoid half-open state. Full wire-format definitions, field validation rules, and encoding rationale appear in the Supplementary Materials (Wire Format and Validation Rules).

Failure semantics. All failures call `HandshakeContext.zeroize()` before propagating the error, emit a typed security event for audit logging, and retain no partial state.

Implementation alignment (artifact snapshot, 2026-01-23). In the current code revision, connection orchestration for the remote desktop/file-transfer path is route-aware and authentication-gated: local discovery can prefer USB-capable host endpoints, control-channel connections are promoted to "connected" only after handshake authentication, discovery advertisements expose an explicit `hs_soa` capability bit for optional SOA negotiation, and active file-transfer routing prioritizes live authenticated sessions before fallback candidates. These implementation surfaces are enumerated in the Supplementary Artifact Map.

3.4 CryptoProvider Protocol

The CryptoProvider protocol defines a unified interface for all cryptographic operations required by the handshake and session layers [21]. This abstraction enables transparent substitution of underlying implementations without modifying caller code.

The `SigningKeyHandle` abstraction resolves the apparent conflict between "private key as Data" and "Secure Enclave keys never leave hardware." Software keys are one variant; hardware-backed keys use a reference or callback, ensuring the protocol interface does not assume exportability.

The protocol mandates `Sendable` conformance, ensuring thread-safe usage across Swift's structured concurrency model. Each provider exposes its tier classification (`nativePQC`, `liboqsPQC`, or `classic`) and active cipher suite, enabling runtime introspection for logging and policy enforcement.

3.5 CryptoSuite Wire Format

A **suite** defines the handshake tuple `(KEM, SIG, AEAD, KDF)` used for the KEM-DEM envelope and transcript binding. Data-plane AEAD is negotiated separately and is fixed to AES-256-GCM in v1 (Supplementary Table S7). Suite identifiers use a compact 16-bit wire ID with reserved ranges for hybrid, PQC, classic, and experimental suites to enable forward compatibility.

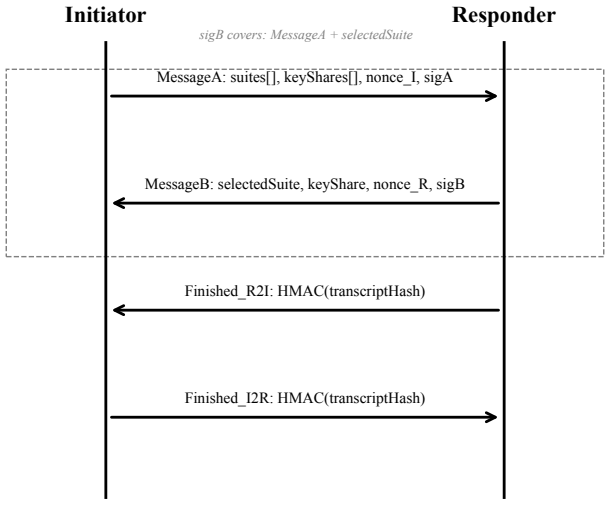

Fig. 1. Handshake sequence with transcript coverage and policy visibility.

The full range table, encoding rules, and X-Wing notes appear in the Supplementary Materials (Suite Identifiers and Wire Format). Unknown suite identifiers are parsed as `unknown(wireId)` for audit visibility and forward compatibility; negotiation rejects unknown values if they appear in `keyShares` or as `selectedSuite`.

3.6 Provider Factory and Selection

The `CryptoProviderFactory` implements deterministic provider selection based on runtime capability detection and policy configuration:

The selection algorithm first queries the environment for native PQC availability and liboqs presence, matches the detected capabilities against the requested policy, instantiates the appropriate provider, and emits a `SecurityEvent` recording the selection decision.

For `preferPQC` policy, the precedence order is `ApplePQCProvider` → `OQSPQCProvider` → `ClassicProvider`. This order is deterministic. The `requirePQC` policy returns an `UnavailablePQCProvider`. It throws on all operations if no PQC implementation is available.

Terminology alignment. We use two orthogonal knobs: provider selection (`preferPQC`/`requirePQC`) and handshake fallback policy (`default`/`strictPQC`). In our evaluation, `default` corresponds to prefer-PQC with evented classic fallback permitted, while `strictPQC` requires PQC with either (i) Compliance Mode (no classic establishment) or (ii) Bootstrap-Assisted Mode (control-only bootstrap plus mandatory PQC rekey).

3.7 KEM-DEM Authenticated Encryption Format

We use a KEM-DEM envelope (KEM → HKDF → AEAD) as a compatibility layer when native HPKE is unavailable, binding role and transcript into the key schedule with explicit length limits for DoS protection. On Apple 26+ platforms, we use native CryptoKit PQC primitives (ML-KEM/ML-DSA) within the same envelope to preserve a single wire format across platforms. Full header/body formats, versioning, and length limits are provided in the Supplementary Materials (KEM-DEM Envelope).

Why not direct RFC 9180 HPKE everywhere (v1)? Three engineering constraints motivate this compatibility layer in v1: (i) a unified transcript/signature binding surface across mixed provider stacks, (ii) strict backward compatibility with deployed v1 bridge behavior, and (iii) deterministic parser-level DoS checks before provider dispatch. When native HPKE is available (Apple 26+), our v2 path uses RFC 9180 HPKE directly; thus the v1 KEM-DEM envelope is transition code, not a claim of new cryptographic primitive design.

4 HANDSHAKE STATE MACHINE

4.1 Design Principles

The handshake subsystem addresses three critical requirements:

1) **Race Condition Prevention:** Concurrent message arrival and timeout expiration must not cause double-resume of continuations.
2) **Sensitive Material Protection:** Ephemeral private keys and shared secrets must be zeroized on all exit paths.
3) **Observability:** All state transitions and failures must emit structured events.

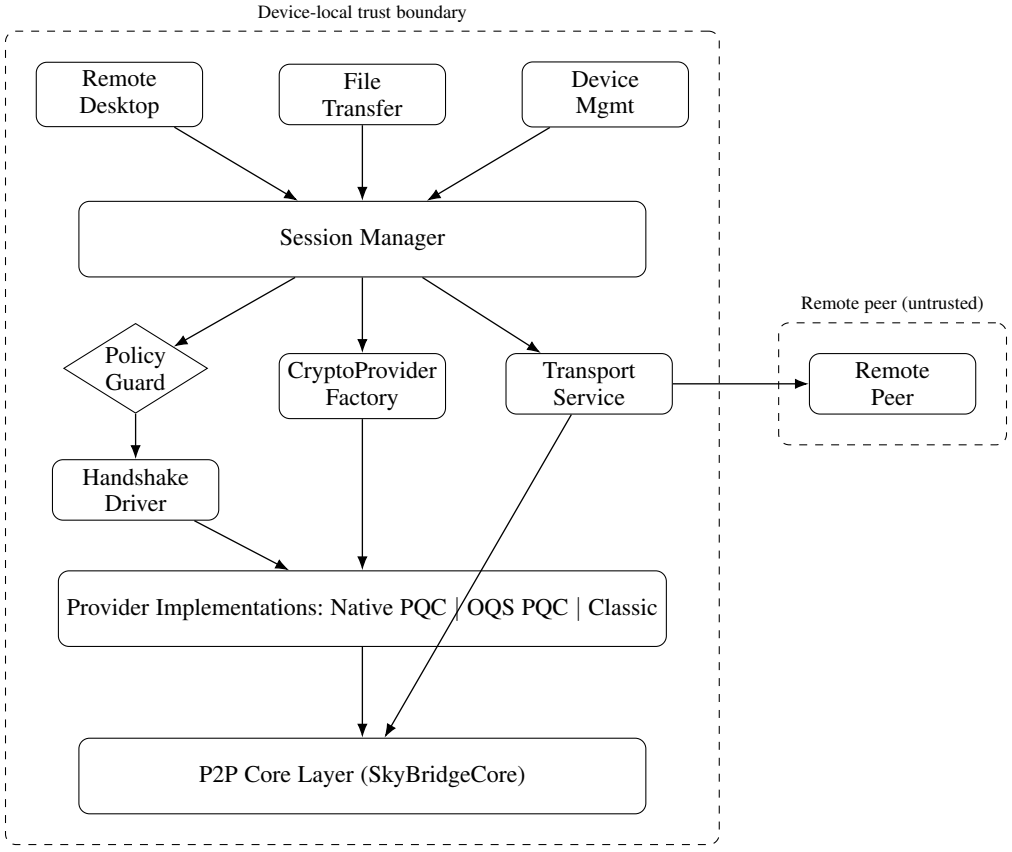

Fig. 2. SkyBridge Compass system architecture showing device-local trust boundary (dashed) and policy guard (diamond).

4.2 Actor-Isolated Architecture

The `HandshakeDriver` actor manages handshake state with compile-time data race prevention:

The `HandshakeContext` actor isolates sensitive cryptographic material:

Reentrancy Considerations. Swift actors permit method reentry at suspension points (`await`). While actor isolation prevents data races, it does not prevent interleaving of method executions. Our `HandshakeDriver` mitigates reentrancy risks by routing completion through `finishOnce(with:)`, buffering early results, canceling timeouts on completion, and advancing state before `await` points.

Await-window limitation. The `handleMessageB` method awaits `cryptoProvider` operations inside the actor. If a new message arrives during this await, the actor could theoretically process it before the current operation completes. We mitigate this by advancing state to `.processingMessageB` before the await, copying immutable inputs, validating a monotonic epoch counter after resumption, and emitting security events on unexpected transitions.

This approach provides structural protection against reentrancy hazards rather than relying on timing assumptions. Formal model checking of actor await-window interleavings is outside the scope; we mitigate and regression-test this class via epoch checks and targeted fault injection (Table 9; Supplementary Table S6).

4.3 State Transitions

Fig. 3 illustrates the handshake state machine for both roles.

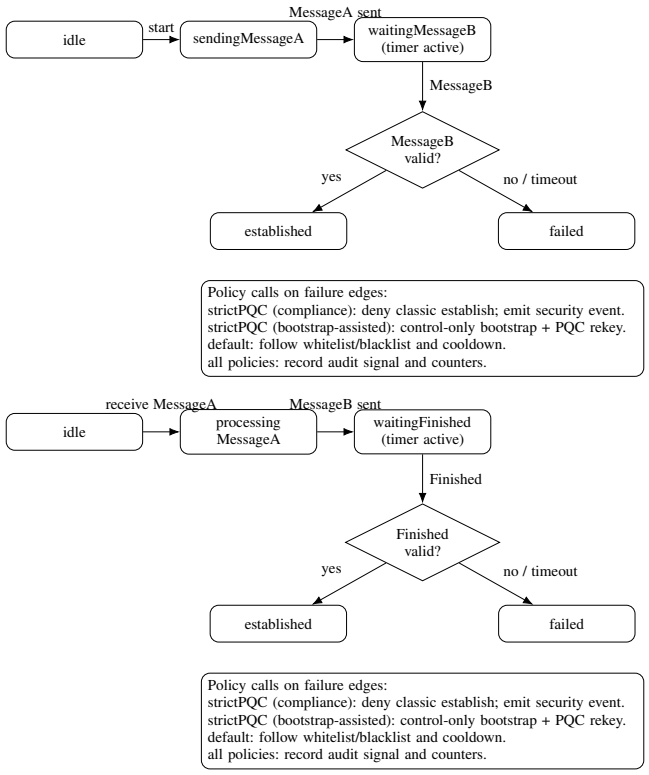

Fig. 3. Handshake state machines for Initiator (top) and Responder (bottom).

4.4 Simultaneous-Open Arbitration (SOA)

To remove nondeterminism when both peers initiate concurrently, we use an optional MessageA extension (SOA TLV) without adding handshake round-trips. The extension carries `soaVersion(1B),initiatorPeerId(32B), targetPeerId(32B),attemptId(16B)`. Arbitration compares the tuple `(initiatorPeerId,attemptId)` with lexicographic *byte* ordering (not string ordering); the smaller tuple wins.

Compatibility is capability-gated (`hs_soa=1` in discovery TXT). If absent, peers stay on the legacy MessageA path. The parser keeps `extensionsRaw` bytes (including unknown TLVs) and transcript/hash binding is computed over the original encoded bytes, so decode/re-encode normalization cannot desynchronize signatures. A local outgoing attempt is superseded only after: (i) MessageA signature verification succeeds, (ii) `targetPeerId==localPeerId`, (iii) `initiatorPeerId==expectedRemotePeerId`, and (iv) the same canonical `pairKey` is selected. Runtime controls combine `single-flight` (one pending winner per `pairKey`), `established guard` (new attempts return `already_connected`), and bounded supersede rate (3 attempts / 60 s). The pending arbitration window is 10 s; established guard is released immediately on disconnect.

4.5 Double-Resume Prevention

The `finishOnce` method provides a single convergence point for handshake completion, handling the race between timeout expiration and message arrival by: (1) canceling the timeout task on any completion, (2) guarding continuation access with immediate nil assignment, and (3) buffering early results for late continuation establishment.

4.6 Secure Zeroization

Swift's standard `Data` and `Array` types use copy-on-write (COW) semantics, which can leave uncontrolled copies of sensitive material in memory. The `SecureBytes` class addresses this by using manual `UnsafeMutableRawPointer` allocation to maintain exclusive ownership, preventing COW-induced secret proliferation. All failure paths explicitly call `context.zeroize()` before error propagation, with `deinit` zeroization as defense-in-depth. On Darwin platforms, zeroization uses platform-supported secure wipe primitives (e.g., `explicit_bzero` or `memset_s`) where available. We avoid exporting secrets as `Data` when possible, but acknowledge that OS-level snapshots, API boundary copies, or privileged inspection can still retain residues; zeroization is a defense-in-depth measure rather than a sole security boundary.

4.7 Transcript Binding and Signature Separation

The protocol uses domain-separated signatures to prevent cross-message and cross-session replay attacks.

Signature A (MessageA) is computed by the Initiator over the MessageA-local fields, binding suite preference, policy intent, and identity context before any responder data is received:

For Signature A, the initiator signs `SkyBridge-A`, protocol version, canonical suite/key-share set, client nonce, capabilities, policy, and initiator identity key. Optional Secure Enclave binding uses `SkyBridge-SE-A`. When SOA extensions are present, the signature/transcript includes the raw extension container bytes (unknown TLVs preserved as-is), not a decoded/re-encoded structure.

Signature B (MessageB): The Responder signs a transcript that commits to MessageA:

For Signature B, the responder signs `SkyBridge-B`, `transcriptA`, selected suite wire ID, responder key share, server nonce, payload hash, and responder identity key. Optional Secure Enclave binding uses `SkyBridge-SE-B`.

Key Derivation: Nonces, transcripts, suite binding, and *directional separation* are included in the KDF, and the salt is transcript-bound (non-empty):

KDF input concatenates the `SkyBridge-KDF` label, suite wire ID, transcript hashes, and nonces. The salt uses the domain string `SkyBridge-KDF-Salt-v1|`, and direction labels (`initiator_to_responder`, `responder_to_initiator`) separate traffic classes.

This structure is designed to ensure: (1) `sigB` commits to `MessageA`, so any tampering breaks verification; (2) domain separators prevent cross-protocol attacks; (3) SE signatures are session-bound and non-replayable.

5 SECURITY MODEL AND GUARANTEES

5.1 Security Model

We assume an active network adversary with full control of the discovery channel (drop, delay, reorder, duplicate, modify, and inject messages) but without breaking standard cryptographic assumptions. The attacker can replay prior handshakes and attempt downgrade by suppressing PQC-related fields. We assume the out-of-band pairing ceremony is trusted and that device key storage (Keychain/Secure Enclave) preserves key integrity. The attacker may compromise a peer after pairing and learn long-term signing key material, but cannot retroactively forge signatures for past sessions. We do not claim forward secrecy for KEM-based suites in v1: ML-KEM encapsulation targets a peer's long-term KEM public key obtained during pairing. Accordingly, post-quantum confidentiality against store-now-decrypt-later adversaries holds only while long-term KEM private keys remain uncompromised; later compromise of the corresponding KEM private key can enable decryption of recorded sessions.

This v1 choice is intentional for migration: it binds PQC establishment to paired long-term KEM identity material that can be provisioned and audited out-of-band, enabling incremental rollout across heterogeneous fleets without introducing an additional interactive key-registration step. To reduce post-compromise exposure while remaining bridge-compatible, we provide v2 suites that mix the static KEM secret with an ephemeral contribution under transcript binding (Proposition P2) and evaluate the incremental overhead (Supplementary Table S22).

We assume the pairing ceremony delivers the correct peer identity key (PIN or visual verification is not bypassed), that pinned identity keys in Keychain/Secure Enclave retain integrity across normal device operation, and that devices remain uncompromised (no root or malware control) during handshake execution.

Under concurrent-open threat conditions, the attacker may inject or replay MessageA variants to try to force cancellation of a valid local attempt. The SOA path treats such attempts as unauthenticated control input until signature verification and peer-binding checks complete; therefore cancellation is only admissible for authenticated, correctly bound attempts within the same canonical `pairKey`.

If assumptions fail. If pairing is subverted and the pinned identity key is replaced, peer authentication is no longer meaningful; the system requires re-pairing and treats the session as untrusted. If device integrity is lost, confidentiality and key secrecy cannot be guaranteed; the system aborts and emits high-severity events such as `identityMismatch` or `handshakeFailed`.

To keep boundary handling operationally explicit, Supplementary Table S38 enumerates boundary-stress cases (pairing social engineering, endpoint compromise classes, and audit-log abuse), their in/out-of-model status, expected event signals, and required operator actions.

5.2 Security Goals and Machine-Checked Proofs

Mechanized symbolic proof. The properties below are *machine-checked* in a Dolev-Yao symbolic model of the handshake, discharged by the Tamarin prover (v1.10.0) [32]; the model is released as `formal/skybridge_v2.spthy`. Every protocol message is sent and received through the network so that Tamarin's built-in active adversary fully mediates the channel (drop, inject, reorder, replay, and synthesize), and the model additionally grants the adversary long-term and KEM private-key compromise via explicit `Reveal` rules. The KEM is modeled symbolically and the v2 ephemeral contribution is included. All 13 lemmas *verify* with no falsifications and no wellformedness warnings; each named property below cites the lemma that discharges it.

The symbolic proof establishes the structural guarantees (authentication, agreement, negotiation integrity, secrecy, forward secrecy, and downgrade resistance) under the standard symbolic abstraction of the primitives. We retain the computational assumptions—KEM IND-CCA, signature EUF-CMA, and HKDF-as-PRF—as the soundness conditions under which the symbolic abstraction is faithful to the implementation; we do *not* claim a computational reduction. Implementation conformance is supported by property-oriented tests that map each proof obligation to concrete code paths and artifacts (Supplementary Table S40); full code-level verification of the Swift implementation is out of scope.

Definitions.

- **D1 (Forced downgrade / forced fallback).** A downgrade occurs if a business-capable session is established with a classic suite when strict policy requires PQC, or if classic fallback is accepted under the default policy without satisfying the local-only fallback gate (Definition D2) and without emitting the corresponding audit event (Property G8).
- **D2 (Local-only fallback gate, default policy).** Fallback is permitted only as an *initiator-side attempt switch* when (i) the session targets an already paired peer (pinned trust record) and (ii) the triggering failure is a locally generated capability/instantiation unavailability detected *pre-send* at the on-wire send boundary (i.e., before emitting the first handshake flight / MessageA in the current attempt); timeout and any network-derived parse/verification failure are fallback-ineligible by definition.
- **D3 (Audit evidence).** The minimal sufficient evidence set is the tuple {event type, timestamp, deviceId, suiteId, failure reason code, policy mode, fallback flag, cooldown remaining} (Property G8).
- **D4 (Bootstrap-Assisted strict recovery gate).** In strict policy, a temporary classic control channel is allowed only for paired KEM-key recovery triggers (`missingKEM`, `staleKEMRecovery`); non-control business messages are blocked until PQC rekey succeeds.

Lemma L1 (Timeout-triggered fallback impossibility). Any migration policy that permits classic fallback on timeout or other unauthenticated network-derived failures is exploitable: under the active attacker of Section 5, the adversary suppresses or delays PQC-path messages past the timeout boundary, so each attempt exposes a reachable classic edge under attacker control and classic establishment is forced.

Corollary (Necessary fallback gate condition). To retain compatibility fallback without attacker-forced downgrade, fallback eligibility must exclude timeout/network-derived failures and be restricted to attacker-non-inducible local causes (Definition D2), or classic fallback must be disabled entirely. This is exactly the contract the model verifies: under strict PQC policy no classic session is ever reachable (`NoClassicUnderStrictPQC`), under the default policy no classic session arises without an explicit, audited downgrade (`NoSilentClassicFallback_Default`), and any classic session implies either a downgrade event or an explicit classic policy (`ClassicRequiresDowngradeOrClassicPolicy`). To rule out vacuity, the model also carries existence lemmas witnessing live PQCv1, PQCv2, and audited-downgrade traces (`Exists_PQCv1_Session`, `Exists_PQCv2_Session`, `Exists_Default_Classic_Downgrade`). Implementation-level traceability anchors are summarized in Supplementary Table S40.

Threat model and security goals (as verified). The adversary $\mathscr{A}$ is the active Dolev-Yao network attacker of Section 5, additionally able to reveal long-term signing and KEM private keys. We require four goals, each discharged by a named Tamarin lemma: *authentication*—$\mathscr{A}$ cannot make a paired endpoint accept an unauthenticated peer (`Injective_Agreement_I_with_R`, `SigB_Authentic_or_Compromised`); *negotiation integrity*—an accepted `selectedSuite` was offered and matches a key share (`NegotiationIntegrity_AcceptedWasOffered`); *downgrade resistance*—classic establishment respects policy as above; and *forward secrecy*—v2 session keys survive long-term-key compromise (`ForwardSecrecy_v2`) whereas v1 does not (`WeakFS_v1`, `V1_LacksForwardSecrecy_Witness`).

Soundness assumptions. The symbolic proof is faithful to the implementation under KEM IND-CCA security for the negotiated KEM (ML-KEM/X-Wing profile), signature EUF-CMA security for the protocol signatures (ML-DSA-65 / Ed25519), and HKDF-SHA256 behaving as a PRF in the extraction/expansion domain used.

Theorem T1 (Authentication). Under the threat model above, no trace lets a paired endpoint complete with a peer that did not run the matching role with agreeing transcript parameters; the agreement is injective (no replay). *Verified:* `Injective_Agreement_I_with_R`, supported by `SigB_Authentic_or_Compromised` (an accepted `sigB` is genuine unless the responder's signing key was revealed).

Theorem T2 (Negotiation integrity and key secrecy). In every trace, the accepted suite was offered by the initiator and bound to a matching key share (`NegotiationIntegrity_AcceptedWasOffered`) and the derived session key is secret against $\mathscr{A}$ in the honest case (`Secrecy_SessionKey_HonestCase`).

Proposition P1 (Policy-gated downgrade). $\mathscr{A}$ cannot force classic establishment under strict PQC policy (`NoClassicUnderStrictPQ`

cannot produce a silent classic session under the default policy (`NoSilentClassicFallback_Default`), and any classic session implies an audited downgrade or an explicit classic policy (`ClassicRequiresDowngradeOrClassicPolicy`).

Proposition P2 (v2 forward secrecy). For v2 sessions, the derived key remains secret even after $\mathscr{A}$ reveals every long-term signing and KEM private key, provided the per-session ephemerals are not revealed—genuine forward secrecy, now *proven* (`ForwardSecrecy_v2`). The model also exhibits a v1 trace where the same long-term compromise breaks v1 secrecy (`V1_LacksForwardSecrecy_Witness`), confirming that the v2 guarantee is strictly stronger than v1's weak forward secrecy (`WeakFS_v1`).

We restate the same guarantees as eight named implementation-facing properties G1–G8, each anchored to the verified lemma that discharges it and to the protocol mechanism that realizes it.

G1 (Negotiation integrity). The selected suite is always one the initiator offered and for which a matching key share exists: `sigB` covers MessageA, so any edit to the offered suite or key-share list breaks verification, and the responder rejects a `selectedSuite` that was not offered. *Verified:* `NegotiationIntegrity_AcceptedWasOffered`.

G2 (Mutual authentication for paired peers). The peer identity is authenticated and bound to the transcript: `identityPubKey` is pinned at OOB pairing, and the MessageA/MessageB signatures over transcript data must verify under the pinned key or the handshake aborts with `identityMismatch`. The agreement is injective. *Verified:* `Injective_Agreement_I_with_R`, `SigB_Authentic_or_Compromised`.

G3 (Session-key secrecy). The shared secret comes from KEM decapsulation (or DH) and is expanded with HKDF over a transcript-bound salt, so an attacker lacking the key material cannot derive the session keys. *Verified:* `Secrecy_SessionKey_HonestCase`. For v1 KEM suites this is conditional on the long-term KEM key (`WeakFS_v1`); v2 adds an ephemeral contribution that removes this dependence (see G3′).

G3′ (v2 forward secrecy). v2 keys survive compromise of all long-term signing and KEM keys as long as the session ephemerals are not revealed; v1 does not. *Verified:* `ForwardSecrecy_v2`, witnessed against v1 by `V1_LacksForwardSecrecy_Witness`.

G4 (Replay resistance). `handshakeId` is derived from fresh nonces and suite identifiers, and duplicates are rejected within a window; replay is precluded by the injectivity of the agreement above. *Verified:* injective `Injective_Agreement_I_with_R`.

G5 (Strict-policy consistency). Strict policy never yields a business-capable classic session: Compliance Mode has no classic edge, and Bootstrap-Assisted Mode allows only a transient control channel with business traffic blocked until PQC rekey. *Verified:* `NoClassicUnderStrictPQC`; validated empirically by the policy downgrade benchmarks (Fig. 4).

G6 (Safe fallback under default policy). Fallback occurs only for an already paired peer and only for a whitelisted, locally generated *pre-send* capability/instantiation error evaluated before MessageA is emitted (Supplementary Table S34); `timeout` and every network-derived failure are excluded by Definition D2, and per-peer cooldown bounds repeated downgrades. Consequently an attacker who can drop, delay, or reorder messages cannot induce a classic session by causing timeouts. *Verified:* `NoSilentClassicFallback_Default`, `ClassicRequiresDowngradeOrClassicPolicy`.

G7 (Legacy acceptance preconditions). Legacy P-256 signatures are accepted only under authenticated pairing or an existing trust record (`LegacyTrustPrecondition`); stranger connections fail, as validated by the precondition test matrix (Table 8).

G8 (Auditability). Every downgrade emits a `cryptoDowngrade` event carrying the minimal sufficient set {event type, timestamp, deviceId, suiteId, failure reason code, policy mode, fallback flag, cooldown remaining} and records it into a per-session, transcript-anchored hash-chain (`AuditTrail`) for in-session tamper-evidence; session keys, nonces, raw signatures, payloads, and user identifiers beyond deviceId are never logged. Long-term retention and remote attestation depend on the operating environment and are out of scope, since a privileged adversary can truncate host logs.

5.3 Complexity and Overhead Bounds

Let S and K denote implementation bounds for offered suites and key shares (`maxSupportedSuites`, `maxKeyShareCount`); currently $S \leq 8$ and $K \leq 2$. Encoding/decoding work for MessageA/MessageB is:

$$T_{\text{codec}} = O(S + K + |M_A| + |M_B|),$$

with constant-time checks on fixed-length fields and linear passes over suite/key-share vectors.

Handshake cryptographic operation counts are:

- **v1 PQC path:** 1× KEM encapsulate + 1× KEM decapsulate + 2 signatures + 2 signature verifies + HKDF key schedule.
- **v2 FS path:** v1 counts plus one ephemeral contribution generation per side and one additional shared-secret composition step (HKDF extract-expand on static+ephemeral input).
- **Classic path:** one DH-style encapsulation/open path (KEM-DEM abstraction), signatures/verifications, and HKDF key schedule.

Closed-form byte overhead between v1 and v2 is dominated by the extra contribution fields. Let c_I denote the initiator contribution carried in MessageA and $c_R^{(v1)}, c_R^{(v2)}$ denote responder-share fields in MessageB for v1 and v2:

$$\Delta|M_A| = 2 + |c_I|, \qquad \Delta|M_B| = |c_R^{(v2)}| - |c_R^{(v1)}|.$$

In current implementation (`X25519` contribution, 32 bytes): $\Delta|M_A| = 34$ bytes and $\Delta|M_B| = 32$ bytes.

6 Implementation

Implementation references (source files, line numbers) are consolidated in the Supplementary Materials (Artifact Map).

6.1 Platform Support Matrix

SkyBridge Compass uses a tiered provider model: a native PQC provider when available (e.g., CryptoKit ML-KEM/ML-DSA), a portable liboqs provider, and a classic provider. Provider selection is deterministic and prefers the strongest available tier; Secure Enclave proof-of- possession is optional and orthogonal to the negotiated suite. The full Apple provider matrix and the cross-platform runtime suite-policy matrix are in the Supplementary Materials (Tables S31 and S33).

TABLE 4
Security Contract (Properties, Enforcement, Evidence).

Property	Enforced at	Evidence
G1 Negotiation integrity	Suite/keyshare validation + transcript-bound `sigB`	Fig. 1, HandshakeDriverTests
G2 Mutual authentication	Identity pinning + signature verification	Supplementary Table S6 (wrong signature), Table 8
G3 Session key secrecy	KEM/DH + HKDF with transcript-bound salt	Property 1, Supplementary Materials (ML-DSA Key Sizes)
G4 Replay resistance	`handshakeId` cache + nonce binding	Property-Oriented Testing (Section 7)
G5 Strict-policy consistency	TwoAttempt policy gate + bootstrap control-only gate + mandatory rekey to PQC	Fig. 4, PolicyDowngradeBenchTests, BootstrapAssuranceTests
G6 Default safe fallback	Whitelist/blacklist + local capability gate + cooldown	Fig. 5, Supplementary Table S6
G7 Legacy acceptance precondition	LegacyTrustPrecondition	Table 8
G8 Auditability	Security event emission + transcript-anchored `AuditTrail` hash-chain	Fig. 6, Table 9

Portable core and outward bindings (contribution C2). The negotiation, downgrade-policy, and audit logic of the contract live in an OS-independent Rust core (`skybridge-core`) that is not tied to any Apple API. The core exposes two coordinated outward binding surfaces (`skybridge-ffi`): a hand-written, panic-safe C ABI (`extern "C"`, header generated by cbindgen) for C/C++/C# P/Invoke, and a UniFFI proc-macro surface for Kotlin and Swift, both thin wrappers over the same core. A non-Apple client therefore drives the identical suite/provider separation, policy-gate, and downgrade-evidence machinery rather than reimplementing the protocol. We deploy this core on macOS and iOS and are consolidating reference Windows (C#/WinUI 3), Android (Kotlin), and Linux (GTK 4) clients onto it. This establishes *mechanism-level* portability; we do not claim measured cross-platform performance parity, and the binding surface deliberately excludes platform-native remote-desktop capture/inject.

6.2 Native PQC Integration

The native provider wraps CryptoKit ML-KEM/ML-DSA APIs [16], [23] when the SDK exposes them, enabling platform-backed PQC suites and HPKE-based encapsulation. We treat this as a deployment instance rather than a protocol dependency; integration details are provided in the Supplementary Materials (Native PQC Integration). Specifically, the Apple provider path does not redefine the protocol contract, does not alter the primary evidence base of the paper, and is evaluated only as an implementation capability mapping under existing negotiation and policy semantics.

6.3 Conditional Compilation Strategy

PQC symbols are gated behind build-time flags to keep the codebase buildable on SDKs that lack PQC types and to avoid compile-time failures. The flagging strategy and build settings are detailed in the Supplementary Materials (Conditional Compilation).

6.4 Security Event Emission

All cryptographic decisions emit structured audit events with backpressure and rate limiting; implementation details are in the Supplementary Materials (Security Event Emission).

6.5 Secure Enclave Integration

For hardware-backed signing, a `SigningCallback` protocol allows Secure Enclave keys without exposing private material. Availability, fallback behavior, and key-type details are provided in the Supplementary Materials (Secure Enclave Integration).

6.6 Signing Key Hierarchy

The system uses two complementary signatures: (1) protocol signatures via the negotiated suite (Ed25519 for classic, ML-DSA-65 for PQC) to bind peer identity to the transcript, and (2) optional Secure Enclave P-256 signatures for device proof-of-possession. Protocol signatures are mandatory; Secure Enclave PoP is optional and provides a hardware-backed signal. Key hierarchy details and verification rules are provided in the Supplementary Materials (Signing Key Hierarchy).

6.7 Pre-Negotiation Signature Algorithm Selection and Two-Attempt Strategy

A fundamental challenge is that `sigA` must be generated before negotiation completes, yet the signature algorithm should match the negotiated suite. We resolve this with pre-negotiation signature selection and a two-attempt strategy. Each attempt is homogeneous (PQC-only or classic-only), and the signature algorithm is chosen based on the offered suites. The initiator first attempts PQC, then falls back to classic only for already paired peers and only for a whitelist of benign, locally generated *pre-send* capability/instantiation errors; timeout, signature verification, identity mismatch, replay, and other network-derived failures never trigger fallback. This policy is enforced by the handshake manager and logged as security events. Detailed error lists and type-level enforcement are provided in the Supplementary Materials (Two-Attempt Strategy). For strict policy deployments, we expose two operator-selectable modes: *Compliance Mode* (hard fail when PQC cannot be established) and *Bootstrap-Assisted Mode* (allow one temporary classic control channel only for paired KEM-key recovery, then require PQC rekey before business traffic).

Compliance Mode is fail-closed and is the reference point for our strict downgrade-resistance claims. Default Mode prioritizes availability and therefore allows a bounded, policy-audited classic fallback for already paired peers under the local-only gate (D2); persistent PQC blocking is treated as an availability attack rather than a silent downgrade.

Fallback boundary conditions (explicit scope). The contract defends against attacker-induced timeouts forcing a downgrade (blocked by the timeout blacklist), rapid downgrade cycling (per-peer cooldown, default 5 min), and signature/identity attacks masquerading as capability failures (verification failures are blacklisted). It does not defend against persistent PQC blocking under Default Mode (an availability-versus-security trade-off for which Compliance Mode is the fail-closed answer) or legitimate PQC-

TABLE 5
Default-policy fallback classes and remote-inducibility boundary.

Reason class	Default-policy outcome
Local capability/instantiation unavailability (paired trust record, pre-send; e.g., local `suiteNegotiationFailed` due to missing paired KEM keyshare)	ALLOW classic fallback + mandatory `cryptoDowngrade` event
Network-derived / after-send failures (`timeout`, remote negotiation mismatch, signature/identity/replay failures)	DENY fallback; emit failure event only

provider failures causing a temporary classic fallback under Default Mode. The 5 min cooldown is a conservative default that bounds cycling while limiting availability impact; it is a tunable operator knob (ablation in Supplementary Table S5), and operators requiring zero classic establishment select strict Compliance Mode.

Attempt state and identifiability. Across attempts only the pinned identity fingerprint and KEM public keys (trust record) and the per-peer cooldown state are shared; nonces, transcript hashes, handshakeId/replay tags, ephemeral key shares, and session keys are not, so each attempt runs a fresh state machine. A passive observer can distinguish a classic fallback attempt from a PQC attempt by wire format; we transmit no explicit attempt counter, and strict Compliance Mode removes classic attempts entirely when capability privacy outweighs compatibility.

Security Events:

Under default policy, classic fallback emits event "cryptoDowngrade" with reason, deviceId, and cooldown fields. Strict Bootstrap-Assisted recovery emits event "handshake_fallback" (bootstrap-assisted mode) with explicit trigger/result fields. This distinguishes strict compliance (fail-closed) traces from strict recovery traces with mandatory PQC rekey.

7 Evaluation

7.1 Experimental Setup

We evaluate SkyBridge Compass along two primary tracks: **security-centric evidence** (fault injection, downgrade suppression, legacy precondition enforcement, and auditability) and **cost-centric evidence** (handshake latency, wire overhead, provider selection overhead, and data-plane throughput).

Environment: All experiments are run on Apple Silicon (M1/M3 class) with macOS 26.x, where CryptoKit PQC is available when the SDK exposes ML-KEM/ML-DSA types [16], [23]. Build configuration uses Release (`-O`) with non-essential logging disabled. Timing uses a monotonic clock (`ContinuousClock`) with 10 warmup runs. The *core gate* (Classic/liboqs/liboqs-v2-FS) uses N=1000 per configuration. Apple PQC/X-Wing are collected in a separate *contrast* pass (N=5000) and are reported as non-gating diagnostics. Reported percentiles (p50/p95/p99) are computed directly from samples; binary outcomes are reported with Wilson 95% confidence intervals.

Repeatability: We support multi-batch repeatability by restarting the benchmark test process for each batch. Supplementary Tables S25–S26 report the number of observed batches (B) and (when $B \geq 2$) mean and 95% confidence intervals across batches. To reproduce multi-batch CIs, re-run the harness with `SKYBRIDGE_BENCH_BATCHES=3-5`.

Reproducibility: We ship an opt-in benchmark suite under `Tests/SkyBridgeCoreTests/` that emits date-stamped CSV artifacts under `Artifacts/` (pinned by `ARTIFACT_DATE=2026-01-23`) and regenerates all evaluation tables/figures from scripts. For a one-shot run, use `Scripts/run_paper_eval.sh`. The runner supports profile-scoped collection via `SKYBRIDGE_BENCH_PROFILE` and `SKYBRIDGE_BENCH_OUTPUT_SET`; contrast outputs are written to dedicated contrast CSV/JSON artifacts. The complete command reference and CSV artifact specifications are provided in the Supplementary Materials (Reproducibility).

iOS Reproducibility: We ship an on-device micro-benchmark in the iOS app (`Settings` → `PQC` → `Self-test/Bench`) that exports schema-v3 JSON artifacts. For the pinned run (`ARTIFACT_DATE=2026-01-23`), each suite uses warmup=10, N=1000, and three batches (B=3). We aggregate exported JSON with `aggregate_ios_microbench.py` into the date-stamped iOS micro-benchmark CSV, then auto-generate the main table and Supplementary device metadata. In this frozen run (iOS/iPadOS 26.2.1), the iOS compatibility-matrix gate passed on both tested device classes (iPad + iPhone), with no schema or negotiation incompatibility observed in the benchmark pipeline.

iOS microbench results: Supplementary Table S21 reports *primitive-level* iOS costs (encap/sign/verify/seal+open), not end-to-end handshake completion; it is therefore not interpreted as a direct substitute for macOS loopback handshake latency. Supplementary Table S20 records per-device metadata for the pinned run.

7.2 Claim Scope and External Validity

We separate *mechanism-level* from *deployment-level* claims. Mechanism-level claims cover negotiation integrity, policy-gated downgrade behavior, deterministic failure semantics, and audit-event completeness under the stated threat model. These claims are tied to the protocol contract and test/formal artifacts, not to Apple-specific APIs. Deployment-level claims cover absolute latency, wire overhead, and provider/runtime effects. They are evaluated only for the reported Apple-platform instance and should not be extrapolated to other OS stacks without remeasurement. To reduce portability ambiguity we ship a cross-platform contract-consistency JSON artifact (Supplementary Table S32) and a dedicated external Linux direct-path run (Supplementary Table S18).

Accordingly, we use the real-network results as external-validity evidence rather than ecosystem-wide performance characterization: the current snapshot includes one STUN-coupled non-overlay direct path, overlay-mediated paths, and a dedicated external Linux direct-path run (`n=50` per payload), and is used to validate directional robustness and failure semantics under deployment constraints.

Evidence overview (main claims). Negotiation integrity and transcript binding rest on the machine-checked model plus protocol and property tests (Section 5; `formal/skybridge_v2.spthy`); policy-gated downgrade correctness on decision-matrix validation, the forced-error policy bench, and the migration-threat harness (Figs. 4, 5; Tables S9, 6); deterministic failure semantics on fault injection and spec-to-event mapping (Supplementary Table S6; Table 9); performance on distributional microbenchmarks and system-level amortization (Table 10; Fig. 10); and external validity on controlled simulation, kernel cross-check, fleet dynamics, and the real-network micro-study (Section 7.7). Supplementary Table S3 provides the full claim-to-evidence map.

Threats to validity. Timing can be affected by runtime noise (scheduler/cache/thermal), so we use Release builds, warmup, distributional reporting, and multi-batch restarts (internal). Loopback time-to-ready is not user QoE, so we report `T_connect`, `T_first_frame`, and `T_total` separately (construct). Absolute costs are Apple-platform-instance results, with cross-stack evidence framed as contract consistency plus targeted Linux direct-path checks (external). Audit-signal fidelity is deterministic spec-to-event conformance rather than statistical, and all artifacts are published for independent reruns (conclusion).

7.3 Security Validation

We validate security properties through fault injection, property-based testing, and audit-signal fidelity analysis. This section presents the key results; detailed test matrices and methodology are provided in the Supplementary Materials (Regression Testing).

Migration-threat harness (baseline contrast). To evaluate migration behavior under active suppression, we run a targeted harness with four metrics: **FDR** (forced downgrade rate), **SDR** (silent downgrade rate), **ECR** (evidence completeness rate for emitted downgrade events), and **PCR** (policy-decision replayability rate from recorded traces). We compare a naive timeout-fallback baseline against SkyBridge policies under a timeout-attack scenario (N=1000). This baseline models a common availability-first glue policy where timeouts are treated as fallback triggers; Lemma L1 applies to any migration strategy that makes timeout/network-derived failures fallback-eligible.

This contrast provides implementation-level evidence for Lemma L1 and its corollary: timeout-eligible fallback is attacker-forcible, while a local-only gate removes that adversarial edge.

Provider/back-end consistency check (non-Apple path). To validate that migration semantics are not tied to Apple-native providers, we rerun the same harness using explicit liboqs and classic provider selection. Supplementary Table S4 reports the provider-matrix results (N=1000 per row).

Fleet-level migration dynamics (heterogeneous capability mix). To quantify migration behavior beyond per-handshake microbenchmarks, we run a controlled fleet simulation with heterogeneous peer capability ratios, per-peer cooldown on/off ablation, and active timeout suppression (N=10,000 sessions per row; peer pool=2,000; timeout attack rate=5%). Supplementary Table S5 reports availability, downgrade exposure, and policy-decision distribution from `Artifacts/migration_fleet_behavior_2026-01-23.csv`.

The fleet-level `mean connect` metric is an in-process policy/attempt completion cost (not external network RTT) and is reported only to compare policy modes under identical harness conditions.

Failure-Mode Robustness. The actor-isolated handshake driver targets deterministic failure semantics; in our fault-injection tests we observed no unhandled errors, double-resume, or sensitive-material leaks. We inject faults including timeout, malformed framing, invalid signatures, out-of-order delivery, and concurrent cancellation (n=1000 per scenario per policy).

Unknown-suite handling: We parse unknown suite identifiers as an explicit `unknown(wireId)` value and reject them if they appear in `keyShares` or as `selectedSuite`. This prevents silent "unknown → classic" coercions that would otherwise weaken downgrade guarantees.

Supplementary Table S6 reports the full per-scenario robustness and observability matrix and event counts (n=1000 per scenario; strictPQC counts recorded separately in `fault_injection_2026-01-23.csv`).

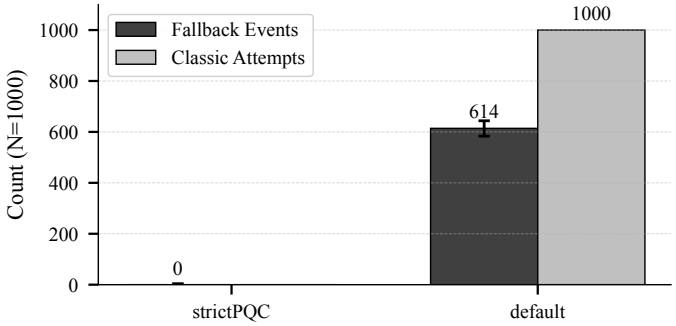

Fig. 4. Policy guard enforces strictPQC Compliance Mode: fallback events per 1000 runs, macOS 26.x, N=1000 per policy. Error bars show 95% Wilson binomial CI.

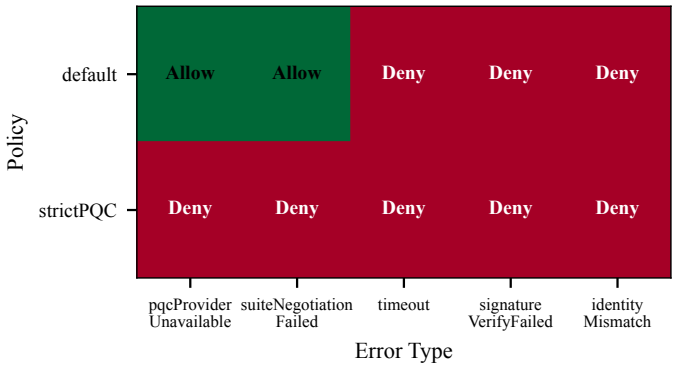

Fig. 5. Downgrade decision matrix (policy × error) with explicit allow/deny semantics, macOS 26.x. Each cell is labeled ALLOW or DENY for grayscale readability.

The delay-within-timeout and delay-exceed-timeout scenarios approximate RTT jitter and loss-induced stalls, while drop and duplicate cases approximate packet loss and reordering. The per-scenario event distributions are recorded in `Artifacts/fault_injection_2026-01-23.csv`.

Downgrade events are quantified separately in the policy bench (Fig. 4) to avoid conflating fallback behavior with transport corruption scenarios.

SOA Arbitration and Compatibility Stress. We add a focused SOA regression bench (`SOAInteroperabilityBenchTests`, `N=100` per scenario) covering simultaneous-open convergence, forged-target rejection, established-guard release, supersede rate limiting, and legacy/new MessageA interoperability. Table 7 summarizes the results; raw rows are stored in `Artifacts/soa_interop_2026-02-22.csv`.

Fig. 4 (dataset `policy_downgrade_2026-01-23.csv`) shows that strictPQC Compliance Mode never emits fallback events even under forced PQC-unavailable errors, whereas default policy shows non-zero downgrade events.

Fig. 5 encodes the explicit downgrade whitelist/blacklist: only PQC-unavailability and local pre-send suite-selection errors may fallback under default policy; strictPQC Compliance Mode denies classic fallback edges (Bootstrap-Assisted strict recovery is a separate control-only path with mandatory PQC rekey).

Fig. 6 combines fault-injection counts with the downgrade-acceptance event from the policy bench to visualize observability

TABLE 6
Migration-threat harness under active timeout suppression (N=1000 per row). Data source: `Artifacts/migration_threat_harness_2026-01-23.csv` produced by `MigrationThreatHarnessBenchTests`.

Configuration	Scenario	FDR	SDR	ECR	PCR	Interpretation
SkyBridge default	Timeout attack	0.00	0.00	1.00	1.00	Timeout does not open classic fallback edge
SkyBridge strictPQC	Timeout attack	0.00	0.00	1.00	1.00	Compliance mode remains fail-closed under suppression
Naive timeout-fallback baseline	Timeout attack	1.00	1.00	0.00	0.00	Attacker can force classic with no replayable audit evidence
SkyBridge default	Local unavailability	1.00	0.00	1.00	1.00	Local-only fallback path is explicit and fully evidenced

TABLE 7
SOA arbitration and compatibility stress results (`ARTIFACT_DATE=2026-02-22`, `N=100` per row).

Scenario	Check focus	N	Pass	Pass rate
Simultaneous open single-flight	Deterministic winner via byte-ordered (`initiatorPeerId`, `attemptId`)	100	100	1.00
Forged target binding guard	Reject unauthenticated cancellation trigger (`targetPeerId` mismatch)	100	100	1.00
Established guard release	`already_connected` while established; immediate reconnect after release	100	100	1.00
Supersede rate limit	Bound supersede to 3/60s; 4th attempt rejected	100	100	1.00
Legacy MessageA compatibility	Missing SOA extension remains valid legacy path	100	100	1.00
Extension passthrough + transcript binding	Unknown TLVs preserved in `extensionsRaw`; SOA-only wire delta = 91 B, no extra RTT	100	100	1.00

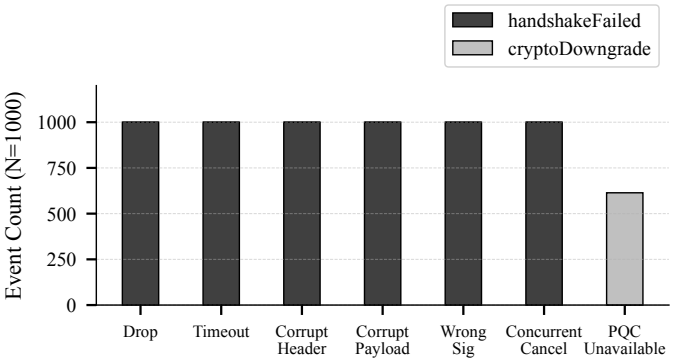

Fig. 6. Failure-mode histogram of security events, macOS 26.x, N=1000 per scenario.

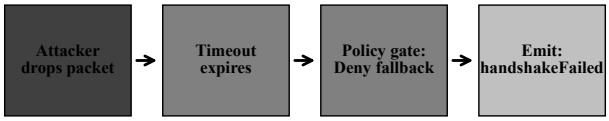

Fig. 7. Audit-signal event trace (representative timeout case) showing attacker action, policy gate, and emitted event.

of failures and downgrades across policy modes.

Property-Oriented Testing. Correctness properties are validated with property-based tests using a reproducible seed (0xDEADBEEF): KEM-DEM round-trip correctness across all providers (100 trials), signature verification integrity (100 payloads per provider), secure zeroization on deallocation, and the state-machine single-resume guarantee under concurrent stress (500 iterations). A standalone Python decoder (`Scripts/wire_format_sanity.py`) provides non-Swift interoperability validation of wire formats.

ML-DSA Key Lifecycle. ML-DSA-65 key generation, storage, and sign/verify operations are validated against FIPS 204 [24]. Size-compliance results are reported in the Supplementary Materials (ML-DSA Key Sizes). All property tests (round-trip, tampering detection, key independence) achieve 100% pass rate. Keys are stored in Keychain with device-local accessibility and synchronization disabled (`kSecAttrSynchronizable=false`).

Legacy Fallback Security. Legacy P-256 signatures are only accepted when a security precondition is satisfied: (1) authenticated out-of-band channel (QR code, PAKE/PIN), or (2) existing TrustRecord with `legacyP256PublicKey`. Pure network stranger connections are rejected to prevent downgrade attacks.

Property G7 (legacy-fallback precondition) is validated with 12 test cases covering all precondition combinations (100% coverage of the precondition state space): a legacy P-256 signature is accepted only when the precondition holds, and pure network strangers are rejected.

Audit-Signal Fidelity. Security events provide deterministic, protocol-level audit signals of failures and policy decisions. Under non-compromised runtime assumptions, the per-session `AuditTrail` hash-chain provides tamper-evident ordering; long-term retention depends on the operating environment. Fig. 7 illustrates a representative timeout trace showing attacker action, policy gate response, and emitted event.

Evaluation approach: We evaluate audit-signal fidelity using a spec-to-event mapping consistency metric, *not* a statistical classifier. Each fault-injection scenario deterministically maps to exactly one expected trace template (event sequence), e.g., timeout → `handshakeFailed=1, cryptoDowngrade=0`. Match rate (MR)=1.00 means all N iterations matched the expected template; spurious rate (SR)=0.00 means no unexpected events were emitted. This is regression consistency against the protocol specification, not ML classification performance.

Table 9 summarizes results across all fault-injection scenarios: all scenario classes achieve 1.00 match and 0.00 spurious rates, confirming correct event semantics.

TABLE 8
Legacy Fallback Precondition Test Results. All tests executed via LegacyFallbackPreconditionTests. Tests validate Property G7: Legacy Fallback Security Precondition.

Scenario	Precondition	Expected Result	Actual Result
Pure network stranger	None	Reject	OK Reject
QR code pairing (verified)	authenticatedChannel	Allow	OK Allow
PAKE pairing (verified)	authenticatedChannel	Allow	OK Allow
Existing TrustRecord with legacy key	existingTrustRecord	Allow	OK Allow
Existing TrustRecord without legacy key	None	Reject	OK Reject
Network discovery	None	Reject	OK Reject
Unverified authenticated channel	None	Reject	OK Reject

TABLE 9
Audit-Signal Fidelity Summary. Match/spurious rates reflect *deterministic* protocol event mapping (each scenario maps to exactly one expected trace template), not statistical classification. Perfect scores indicate regression consistency against the protocol specification.

Scenario Class	Expected Signal	N	Match (MR)	Spurious (SR)
Drop/Timeout (default)	handshakeFailed=1, cryptoDowngrade=0	3000	1.00	0.00
Drop/Timeout (strictPQC)	handshakeFailed=1, cryptoDowngrade=0	3000	1.00	0.00
Corrupt/Wrong Sig	handshakeFailed=1, cryptoDowngrade=0	6000	1.00	0.00
Ordering Benign	handshakeFailed=0, cryptoDowngrade=0	6000	1.00	0.00
PQC unavailable (default)	cryptoDowngrade=1	1000	1.00	0.00
PQC unavailable (strictPQC)	cryptoDowngrade=0	1000	1.00	0.00

Due to space constraints, the complete regression test matrix, TLV encoding validation, and implementation component details are provided in the Supplementary Materials (Regression Testing and Artifact Map).

7.4 Resource Amplification and Throttling

We separate adversarial cost vectors into (i) pre-authentication parser cost and (ii) post-authentication retry/cooldown pressure. Pre-auth cost is bounded by strict field-length checks, overflow-safe parsing, and bounded key-share cardinality before cryptographic operations are invoked. Post-auth pressure is bounded by per-peer fallback cooldown and rate-limited security event emission.

These controls are intended to bound resource amplification from malformed inputs and retry storms without introducing silent downgrade paths. We do not claim Internet-scale saturation resistance from this revision alone; adversarial burst-throughput characterization remains future work.

7.5 Performance Benchmarks

Table 10 consolidates *core gate* performance metrics. We measure end-to-end handshake completion time using an in-memory loopback transport to isolate cryptographic and state-machine overhead. Each core configuration reports distributional statistics after 10 warmup runs (N=1000 per configuration). Apple PQC/X-Wing are reported separately as non-gating contrast data.

v1/v2 Bridge Overhead. To isolate the protocol-v2 cost, we run a controlled liboqs v1/v2 bridge comparison with identical hardware/runtime settings (N=1000, warmup=10). Supplementary Table S22 reports mean/p95 latency, RTT, and payload-handshake bytes. Consistent with the wire-format analysis in Section 5.3, v2 adds a fixed +75 B payload-handshake overhead (MessageA +43 B, MessageB +32 B), while latency impact remains in the low-millisecond range.

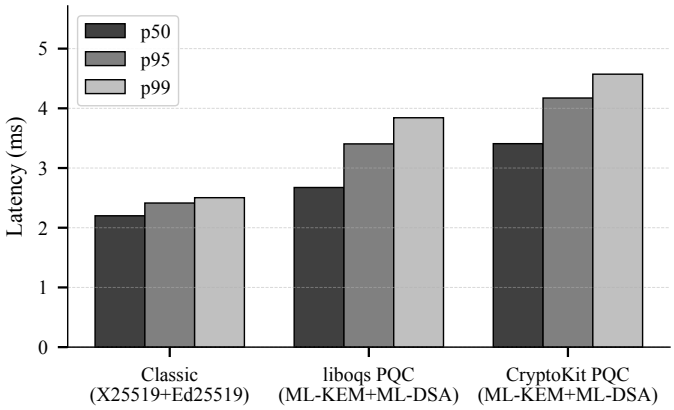

Fig. 8. Handshake latency percentiles for the core gate configurations (Classic, liboqs PQC, liboqs PQC v2 FS), warmup=10 and N=1000 per configuration. Data source: `handshake_bench_2026-01-23.csv`. Values align with Table 10 and Supplementary Table S12.

Latency Distribution. Fig. 8 shows latency percentiles across configurations. Classic handshakes remain near 2.232 ms (2.503 ms at p99). The liboqs v2 FS configuration remains around 4.572 ms at p99. Extended statistics (mean/p50/p95/p99, std) are available in Supplementary Tables S12–S13 (reported in the supplementary PDF).

Wire-Format Breakdown. The handshake payload size increases from Classic (687 B) to PQC (12002 B for liboqs; 12077 B for liboqs v2 FS in the core gate), dominated by ML-DSA signatures (3309 B) and ML-KEM ciphertexts (1088 B). Apple PQC/X-Wing payloads are reported in supplementary contrast artifacts. Fig. 9 shows the per-field breakdown. **Path-MTU implication.** The payload-only core-gate PQC handshake (12002 B for liboqs; 12077 B for liboqs v2 FS) spans multiple transport segments under common MTUs. In our design, handshake mes-

TABLE 10
Performance Summary. All benchmarks on Apple Silicon (M1/M3), macOS 26.x, N=1000 iterations after 10 warmup runs. Wire Size counts payload-only handshake bytes (MessageA + MessageB + 2×Finished); loopback wire sizes including transport overhead are reported separately in Table 2 and Supplementary Table S24. Data-plane AEAD is fixed to AES-256-GCM in v1, so throughput is independent of the negotiated handshake suite; throughput measured post-handshake on 1 MiB payloads.

Configuration	Handshake Latency		RTT		Wire Size	Throughput
	mean (ms)	p95 (ms)	p50 (ms)	p95 (ms)	(bytes)	(GB/s)
Classic	2.23	2.42	0.47	0.53	687	3.7
liboqs PQC	2.74	3.40	0.82	1.37	12,002	3.7
liboqs v2 FS	3.48	4.18	1.15	1.70	12,077	3.7

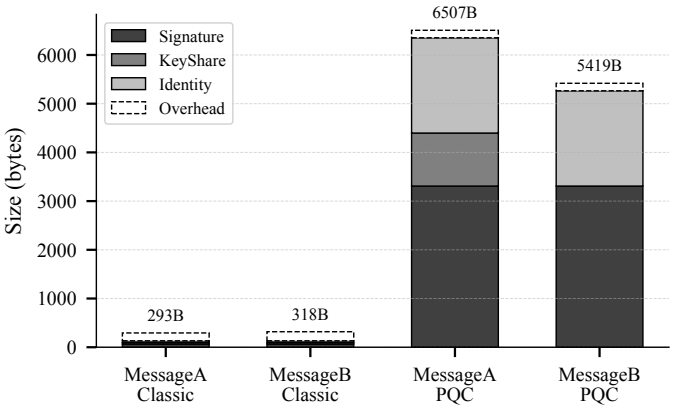

Fig. 9. Handshake message size breakdown (signature, keyshare, identity fields, framing overhead) for MessageA/MessageB across Classic and PQC configurations. Counts reflect MessageA/B payloads only. The *payload-only* handshake size adds two 38 B Finished frames (accounted for in Table 10); detailed field sizes are reported in Supplementary Table S14. Data source: `message_sizes_2026-01-23.csv`.

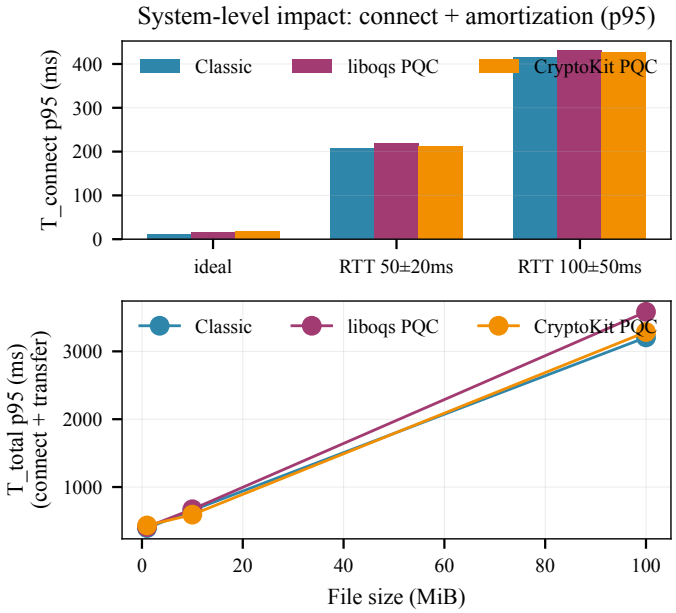

Fig. 10. System-level impact and amortization (p95). Top: `T_connect` across RTT+jitter conditions. Bottom: total session time (`T_total` = connect + transfer) versus file size under RTT 50±20 ms, showing that one-time handshake overhead is amortized by larger transfers. Data from `Artifacts/system_impact_2026-01-23.csv`.

sages run over stream transports (TCP/QUIC stream), so segmentation/reassembly is delegated to transport rather than custom handshake-layer fragmentation; the practical risk is latency inflation on lossy paths, which is why we report controlled impairment, kernel-level, and real-network evidence in Section 7.7.

Note: In the `2026-01-23` snapshot, X-Wing hybrid KEM adds 128 B to MessageA's keyshare (1216 B vs 1088 B for ML-KEM-768). Measured handshake sizes are reported in Supplementary Table S14 (`MessageA.XWing/MessageB.XWing`).

Data-Plane and Provider Selection. Post-handshake symmetric AEAD (AES-256-GCM) throughput reaches 3.7 GB/s for 1 MiB payloads on Apple Silicon, with CPU cost of 0.25 ns/byte. In v1 the data-plane AEAD is fixed to AES-256-GCM and recorded separately from the handshake suite; thus throughput is independent of the negotiated KEM/signature suite. Provider selection overhead is sub-microsecond for cached paths (0.17 μs p50) and under 6 μs even for fallback scenarios. Detailed breakdowns are provided in the artifact CSV files.

7.6 System-level Impact and Amortization

The preceding microbenchmarks isolate handshake and cryptographic costs. To align evaluation with the *remote desktop / file transfer* framing, we additionally measure user-visible, session-level metrics and quantify how one-time handshake cost amortizes over longer sessions and larger transfers.

Metrics. We report `T_connect` (connect start to handshake established, including Finished key confirmation and event emission), `T_first_frame` (to the first decrypted remote-desktop frame at the rendering path), and `T_total` (to completion of a bulk transfer over the negotiated session keys). Because the handshake runs once per session, its low-millisecond cost (Table 10) is a one-time addend to `T_total`: for any non-trivial remote-desktop session or multi-MiB transfer it falls below measurement noise, which is precisely why the deployment can afford the contract's deterministic, auditable negotiation.

Method. We reuse the production `HandshakeDriver` and negotiated `SessionKeys`. The workload is one encrypted "first frame" plus bulk transfers (1–100 MiB) over a reliable stream with a deterministic RTT+jitter channel model; bulk payloads are rate-limited (MiB/s) to avoid per-chunk RTT artifacts, and the date-stamped system-impact CSV artifact is generated by `SystemImpactBenchTests`. *Results:* Supplementary Table S11 reports p50/p95 `T_connect` and `T_total` results under the controlled RTT+jitter profiles.

Traffic Analysis Mitigation (SBP2 Padding). To reduce attempt identifiability on the wire, SkyBridge optionally applies a constant-size framing layer (SBP2) that buckets framed payloads into powers-of-two-sized envelopes. Here, **SBP1** denotes *handshake-frame padding* (MessageA/MessageB/Finished are padded to fixed buckets before transport), while **SBP2** denotes

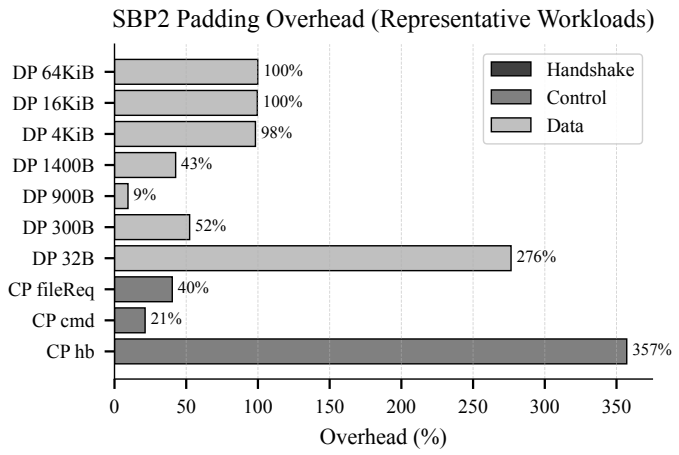

Fig. 11. SBP2 padding overhead across representative handshake, control, and data-plane workloads (macOS 26.x). Overhead is computed as $(\texttt{paddedBytes}/\texttt{rawBytes} - 1) \times 100\%$ aggregated per label over a deterministic trace (N=1000 handshake iterations; data-plane workload uses encrypted control messages and large-frame mixes). Generated from `Artifacts/traffic_padding_2026-01-23.csv`.

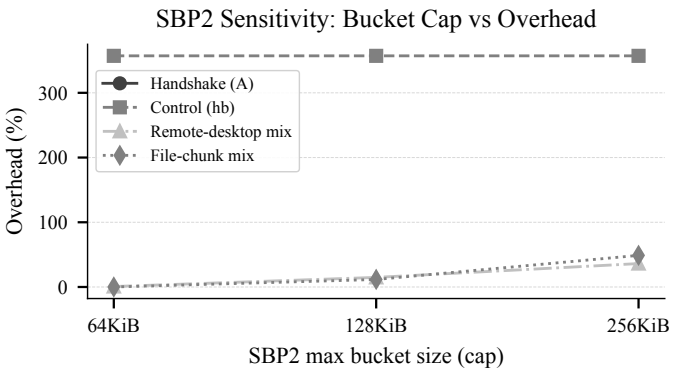

Fig. 12. SBP2 bucket-cap sensitivity on macOS 26.x. Lower over-cap rate means broader padding coverage across 64 KiB, 128 KiB, and 256 KiB caps for remote-desktop and file-transfer mixes.

traffic framing and padding applied to all framed payloads (control and encrypted data-plane) with a per-frame header and bucketed padding up to a configured cap. We measure SBP2 behavior on a real protocol trace: an end-to-end handshake (SBP1 + SBP2 enabled) on an in-memory loopback transport, followed by bidirectional AES-GCM–encrypted data-plane frames derived from the negotiated session keys. Supplementary Table S15 reports the resulting quantization behavior and overhead; the full per-label bucket histograms are included in `Artifacts/traffic_padding_2026-01-23.csv`.

Sensitivity Study (Bucket Cap). We vary the maximum bucket size for SBP2 (*cap* = 64 KiB, 128 KiB, 256 KiB) and re-run the same deterministic protocol trace. We report two overhead definitions: *all-frames overhead* (includes passthrough frames) and *padded-only overhead* (restricted to frames that entered bucket padding), alongside *over-cap rate* (fraction of frames whose framed payload exceeds cap) and *bucket entropy* (Shannon entropy of padded bucket sizes per label). When over-cap is high, all-frames overhead may approach zero while padded-only overhead remains non-zero; this is a cap tradeoff, not a measurement bug. Supplementary Table S16 summarizes this overhead–privacy tradeoff and distinguishes remote-desktop-like vs file-chunk-like large-frame distributions.

Operational decision guide (rule-of-thumb). For interactive remote-desktop-first workloads, prefer SBP2 disabled or 64 KiB cap. For mixed control + moderate transfer workloads, 128 KiB is a balanced default. For bulk-transfer-heavy, privacy-prioritized workloads, 256 KiB improves padding coverage at the cost of higher bandwidth overhead.

AEAD selection (v1). Handshake payloads and data-plane traffic both use AES-256-GCM in v1; Supplementary Table S7 summarizes the handshake-negotiated vs fixed data-plane mapping.

7.7 Network Impairments: Controlled, Kernel-level, and Real-network Evidence

To reduce external-validity ambiguity, we report a three-layer evidence stack with one shared profile source (`Scripts/netem_profiles.json`): (i) controlled in-process simulation, (ii) kernel-level emulation cross-check (`dummynet`/`netem`), and (iii) real-network micro-study.

Controlled simulation (in-process). We simulate four network conditions with a retry-aware model (max 2 retries) to isolate protocol logic under reproducible loss/jitter/reorder perturbations. Loss is Bernoulli per packet; jitter is sampled uniformly from $\pm$J ms around base RTT; reordering is modeled as added random delay (50–150 ms) on 10% of packets.

Controlled-simulation summary: completion is ≥ 0.9989 (min), p95 latency spans 251–1033 ms, and $E_{\texttt{handshakeFailed}} \leq 11$ per 10k runs across the reported profiles/suites. Supplementary Table S8 reports the full controlled-simulation table.

Kernel-level cross-check. Using `dnctl/pfctl` (macOS dummynet) and `tc netem` (Linux) with aligned profile IDs, a kernel-emulation script produces the outputs summarized in Supplementary Table S19. Reorder is reported only for netem (dummynet reorder cells are *n/a*). Because dummynet with localhost endpoints may under-shape absolute delay on some macOS setups, delay-scale reading is anchored by netem and controlled simulation, while cross-tool completion robustness remains the primary check.

Real-network micro-study. We collect STUN path telemetry and TCP end-to-end timing for payload-only handshake baselines (classic 687 B; core-gate PQC 12002 B for liboqs and 12077 B for liboqs v2 FS; Apple contrast 12009 B where available; artifact source `Artifacts/realnet_*`), reported in Supplementary Table S17. The snapshot has one direct label (`wifi_direct`, remote EC2 over TCP 443) and two overlay labels (`ec2_ssh_tunnel`, `hotspot_ssh_tunnel`); direct and overlay paths are reported separately to distinguish external direct-path evidence from tunnel-mediated behavior under restrictive inbound NAT/firewall conditions. The harness sends a `4B length-prefix + payload` and payload-byte labels exclude the prefix. A dedicated direct-path EC2 Linux run (no tunnel, `n=50` per payload) appears in Supplementary Table S18, and measured Ubuntu direct/tunnel rows plus static Android/Ubuntu contract-consistency rows are summarized in Supplementary Table S23.

8 Limitations and Future Work

8.1 Current Limitations

1) **Platform scope:** Main performance numbers are Apple-centric; non-Apple deployments require remeasurement under the same artifact and workload controls.
2) **Secrecy scope:** v1 static-recipient KEM deployments do not provide the same secrecy semantics as the v2 hybrid path (Supplementary Table S22).
3) **External validity:** Real-network evidence remains small-scale and does not yet cover broad mobility and ISP diversity.

8.2 Future Directions

Priority extensions are broader Android/Ubuntu replication, ephemeral-ephemeral secrecy designs beyond v2, and cross-peer audit attestation.

9 Conclusion

We presented an auditable crypto-agile migration contract for P2P authenticated key exchange: it separates suites from providers, confines classic fallback to a local-only pre-send path so timeouts cannot force a downgrade, and emits replayable downgrade evidence. The contract's guarantees—authentication, negotiation integrity, key secrecy, downgrade resistance, and genuine v2 forward secrecy—are machine-checked in a Dolev-Yao Tamarin model with an active, key-compromising attacker (13/13 lemmas verified). The contract is realized in a portable Rust core with C-ABI and UniFFI bindings, answering the "generalize beyond Apple" concern at the mechanism level, and its one-time handshake cost amortizes over long remote-desktop sessions and large transfers. Reported latencies are deployment-instance measurements and should be re-established on non-Apple targets.

Acknowledgment: This work received no external funding.

Data and Artifact Availability

Artifacts are available at github.com/billlza/Skybridge-Compass (commit `6576e954a5bb`); frozen source archives and checksums are documented in the Supplementary Materials.

Conflict of Interest: The author declares no competing interests.

References

[1] Bluetooth SIG, "Bluetooth Core Specification v5.4," 2023.
[2] Apple Inc., "Continuity," Apple Platform Security Guide, 2024.
[3] T. Taubert and C. A. Wood, "SPAKE2+, an Augmented PAKE," RFC 9383, IETF, Sept. 2023.
[4] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, IETF, 2018.
[5] J. Iyengar and M. Thomson, "QUIC: A UDP-Based Multiplexed and Secure Transport," RFC 9000, IETF, 2021.
[6] E. Barker and A. Roginsky, "Transitioning the Use of Cryptographic Algorithms and Key Lengths," NIST SP 800-131A Rev. 2, 2019.
[7] E. Barker et al., "Considerations for Achieving Cryptographic Agility: Strategies and Practices," NIST Cybersecurity White Paper (CSWP) 39, Dec. 2025, doi: 10.6028/NIST.CSWP.39.
[8] T. Perrin, "The Noise Protocol Framework," 2018.
[9] NIST, "Post-Quantum Cryptography Standardization," 2024. Accessed: Dec. 2025.
[10] Signal Foundation, "PQXDH Key Agreement Protocol," Signal Technical Documentation, 2023.
[11] Cloudflare, "Post-Quantum Cryptography Goes GA," Sept. 2023. Accessed: Dec. 2025.
[12] P. Schwabe, D. Stebila, and T. Wiggers, "Post-Quantum TLS Without Handshake Signatures," in Proc. ACM CCS 2020, pp. 1461–1480.
[13] E. Alkim, L. Ducas, T. Pöppelmann, and P. Schwabe, "Post-Quantum Key Exchange—A New Hope," in Proc. USENIX Security 2016, pp. 327–343.
[14] J. Bos et al., "CRYSTALS—Kyber: A CCA-Secure Module-Lattice-Based KEM," in Proc. IEEE EuroS&P 2018, pp. 353–367.
[15] L. Ducas et al., "CRYSTALS—Dilithium: A Lattice-Based Digital Signature Scheme," IACR Trans. CHES, vol. 2018, no. 1, pp. 238–268.
[16] Apple Inc., "Quantum-secure cryptography in Apple operating systems," Apple Platform Security, Jan. 2026. Accessed: Feb. 2026.
[17] D. Beyer et al., "Software Model Checking," in Handbook of Model Checking, Springer, 2018.
[18] B. Beurdouche et al., "A Messy State of the Union: Taming the Composite State Machines of TLS," in Proc. IEEE S&P 2015, pp. 535–552.
[19] Apple Inc., "Swift Concurrency," The Swift Programming Language, 2024.
[20] E. Rescorla and N. Modadugu, "Datagram Transport Layer Security Version 1.2," RFC 6347, IETF, Jan. 2012.
[21] M. Green and M. Smith, "Developers Are Not the Enemy!: The Need for Usable Security APIs," IEEE Security & Privacy, vol. 14, no. 5, pp. 40–46, Sept./Oct. 2016.
[22] K. Bhargavan, C. Fournet, M. Kohlweiss, A. Pironti, and P.-Y. Strub, "Implementing TLS with Verified Cryptographic Security," in Proc. IEEE S&P 2013, pp. 445–459.
[23] Apple Inc., "CryptoKit Framework," Apple Developer Documentation, 2025. Accessed: Dec. 2025.
[24] NIST, "Module-Lattice-Based Digital Signature Standard," FIPS 204, Aug. 2024.
[25] D. McGrew, "Achieving Crypto Agility," in Proc. RSA Conference, 2019.
[26] R. Barnes, K. Bhargavan, B. Lipp, and C. Wood, "Hybrid Public Key Encryption," RFC 9180, IETF, Feb. 2022.
[27] Apple Inc., "Get ahead with quantum-secure cryptography," WWDC 2025 Session 314, June 2025. Accessed: Dec. 2025.
[28] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," FIPS 203, Aug. 2024.
[29] NIST, "Recommendations for Key-Encapsulation Mechanisms," SP 800-227, Final, Nov. 2025.
[30] K. Cremers, M. Horvat, J. Hoyland, and S. Scott, "A Comprehensive Symbolic Analysis of TLS 1.3," IACR ePrint 2020/1044, 2020.
[31] P. Schwabe, D. Stebila, and T. Wiggers, "KEMTLS: Authenticated Key Exchange without Signatures in TLS 1.3," 2020.
[32] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The Tamarin Prover for the Symbolic Analysis of Security Protocols," in Proc. CAV 2013, LNCS 8044, pp. 696–701.
[33] M. Barbosa, D. Connolly, J. Duarte, A. Kaiser, P. Schwabe, K. Varber, and B. Westerbaan, "X-Wing: The Hybrid KEM You've Been Looking For," IETF Internet-Draft, draft-connolly-cfrg-xwing-kem-09, Jan. 2026.
[34] D. J. Bernstein, "Curve25519: New Diffie-Hellman Speed Records," in Proc. PKC 2006, LNCS 3958, Springer, 2006, pp. 207–228.
[35] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-Speed High-Security Signatures," J. Cryptographic Engineering, vol. 2, no. 2, pp. 77–89, 2012.
[36] H. Krawczyk, "SIGMA: The 'SIGn-and-MAc' Approach to Authenticated Diffie-Hellman and Its Use in the IKE Protocols," in Proc. CRYPTO 2003, LNCS 2729, pp. 400–425.
[37] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, IETF, May 2010.
[38] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST SP 800-38D, Nov. 2007.
[39] N. Aviram et al., "DROWN: Breaking TLS Using SSLv2," in Proc. USENIX Security 2016, pp. 689–706.
[40] D. Adrian et al., "Imperfect Forward Secrecy: How Diffie-Hellman Fails in Practice," in Proc. ACM CCS 2015, pp. 5–17.
[41] C. Meyer and J. Schwenk, "Lessons Learned From Previous SSL/TLS Attacks—A Brief Chronology of Attacks and Weaknesses," IACR Cryptology ePrint Archive, Report 2013/049, 2013.
[42] K. Kwiatkowski, P. Kampanakis, B. Westerbaan, and D. Stebila, "Post-quantum hybrid ECDHE-MLKEM Key Agreement for TLSv1.3," IETF Internet-Draft, draft-ietf-tls-ecdhe-mlkem-05, May 2026.
[43] D. Connolly, "ML-KEM Post-Quantum Key Agreement for TLS 1.3," IETF Internet-Draft, draft-ietf-tls-mlkem-07, Feb. 2026.